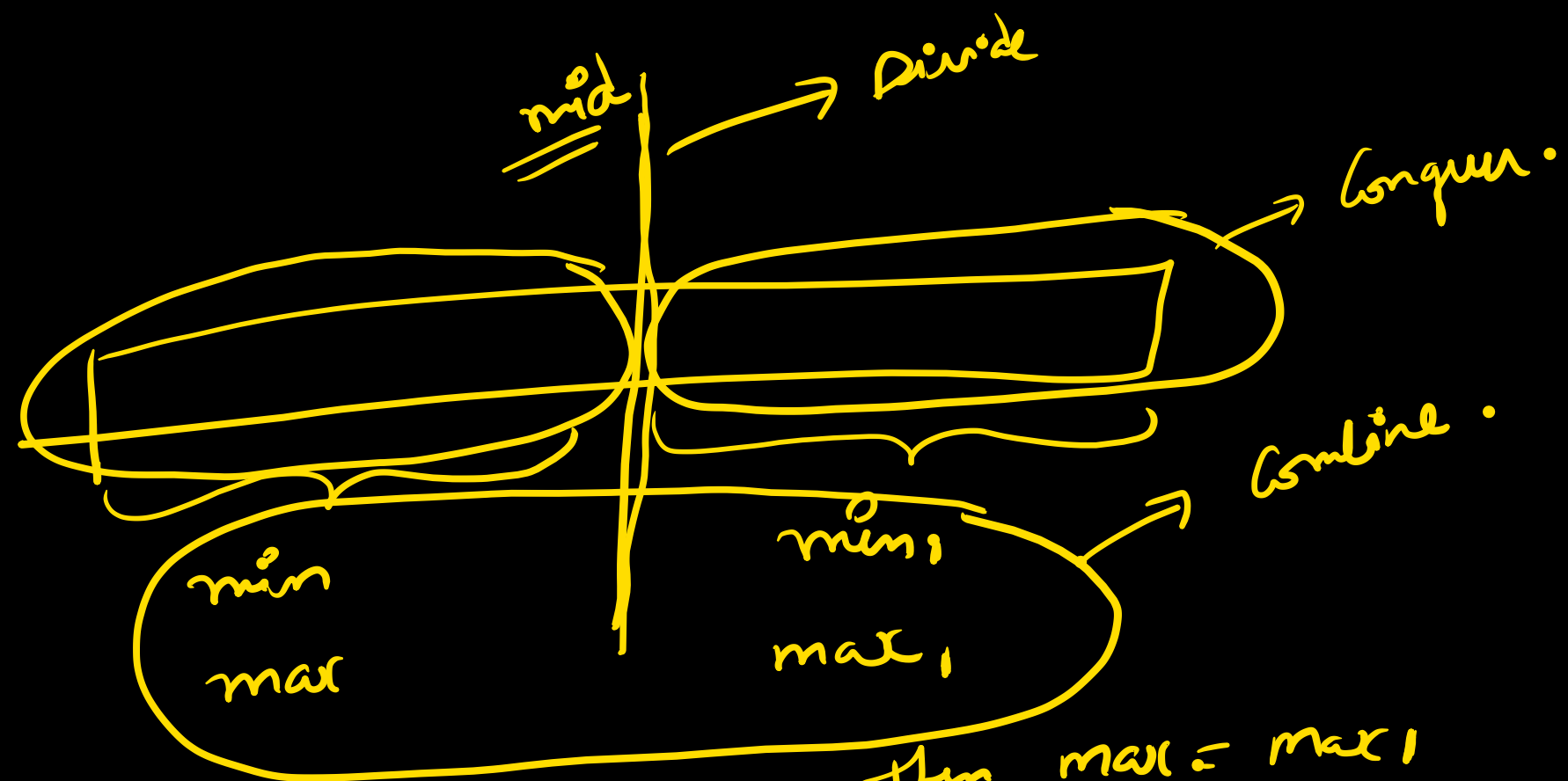




$\max < \max_1$ then $\max = \max_1$

$\min > \min_1$ then $\min = \min_1$

Algorithm $\text{maxmin}(i, j, \text{max}, \text{min})$

```

{
  if ( $i == j$ ) then  $\text{max} = \text{min} = a[i];$ 
  else if ( $i == j - 1$ ) then
  {
    if ( $a[i] < a[j]$ ) then
    {
       $\text{max} = a[j]; \text{min} = a[i];$ 
    }
    else
    {
       $\text{max} = a[i]; \text{min} = a[j];$ 
    }
  }
}

```

Annotations:

- $i == j$: 0 comparisons, Small $\Rightarrow$ Size = 1
- $i == j - 1$: 2 divisions, 1 comparison, Small $\Rightarrow$ Size = 1

else

```

{
   $\text{mid} = \lfloor \frac{i+j}{2} \rfloor$ 
   $\text{maxmin}(i, \text{mid}, \text{max}, \text{min})$ 
   $\text{maxmin}(\text{mid}+1, j, \text{max}, \text{min})$ 
  if ( $\text{max} < \text{max}_1$ ) then  $\text{max} = \text{max}_1$ 
  if ( $\text{min} > \text{min}_1$ ) then  $\text{min} = \text{min}_1$ 
}

```

Annotations:

- $\text{mid} = \lfloor \frac{i+j}{2} \rfloor$: divide.
- $\text{maxmin}(i, \text{mid}, \text{max}, \text{min})$ and $\text{maxmin}(\text{mid}+1, j, \text{max}, \text{min})$: conquer.
- Both problems are divided in 1 or 2 size problem recursive.
- Even divided into smaller problems till they can be solved without dividing.
- Diagram: A yellow oval with a vertical line through it. The left side is labeled "max min" and the right side is labeled "max min".
- Bottom: (2).

a 1 2 3 4 5 6 7 8 9
 22 13 -5 -8 15 60 17 31 47

1, 9, 60, -8 ⑨

max min
1, 5, 22, -8 ⑤

max min
6, 9, 60, 17 ⑧

1, 3, 22, -5 ③

④
4, 5, 15, -8
2

6, 7, 60, 17 ⑥
2

8, 9, 47, 31 ⑦
2

1, 2, 22, 13 ①
2

3, 3, -5, -5 ②
1

max
min
22, 13
..

max
min
-5, -5
..

Size = 1 0 Comp
Size = 2 1 Compar
Size $\geq$ 3 2 Comp.

$C(n)$ is number of Comparisons required to find maximum
on a size of ' n '.

$$C(n) = \begin{cases} C(n/2) + C(n/2) + 2 & n > 2 \\ 1 & n = 2 \\ 0 & n = 1 \end{cases}$$

$$\text{If } C(n) = 2C(n/2) + 2$$

$$= 2(2C(n/4) + 2) + 2$$

$$= 2^2(C(n/2^2)) + 2^2 + 2$$

$$= 2^2(2C(n/2^3) + 2) + 2^2 + 2$$

$$= 2^3 C(n/2^3) + 2^3 + 2^2 + 2^1$$

$$\vdots$$

$$= 2^K T(n/2^K) + 2^K + 2^{K-1} + \dots + 2$$

$$\text{Let } n/2^K = 2 \Rightarrow 2^K = n/2$$

$$= n/2 * T(2) + \frac{2(2^K - 1)}{2 - 1}$$

$$= \frac{n}{2} + 2\left(\frac{n}{2} - 1\right) = \left(\frac{3n}{2} - 2\right)$$

$\left(\frac{3n}{2} - 2\right)$ is the best, average and worst case no. of comparisons.

No method can give better comparisons than ~~Divide~~ divide and conquer max min.

Disadv: In terms of space, maxmin is worse

than straight forward algo because it requires stack space for recursion.

Space complexity:-



$$2 \quad n/2^k = 1$$
$$\Rightarrow k = \log n$$

Time Complexity for DandC max min.

$$T(n) = \begin{cases} d(1), & n=1 \\ O(1), & n=2 \\ T(n/2) + T(n/2) + C \rightarrow & n > 2 \end{cases}$$

$$\frac{TC}{O(n)}$$

$$\frac{SC}{O(\log n)}$$

Comp:

$$\frac{3n}{2} - 2$$

$$T(n) = 2T(n/2) + C$$

$f(n)$

(C)

$$n^{\log_b a}$$

$$n^{\log_2 2}$$

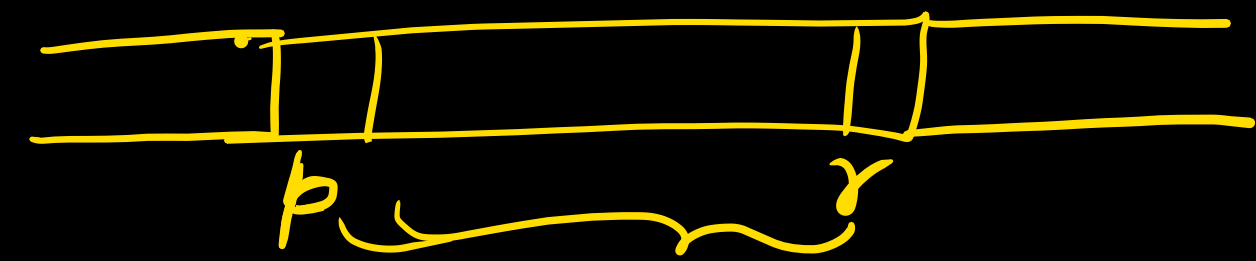
(n) ✓✓

$$\underline{\underline{O(n)}}.$$

one more example on DandC, merge sort

DandC.

merge_sort (A, p, r)



Small
size $\Rightarrow$

if (p < r) $\rightarrow$ number ~~of~~ elements are ≥ 2

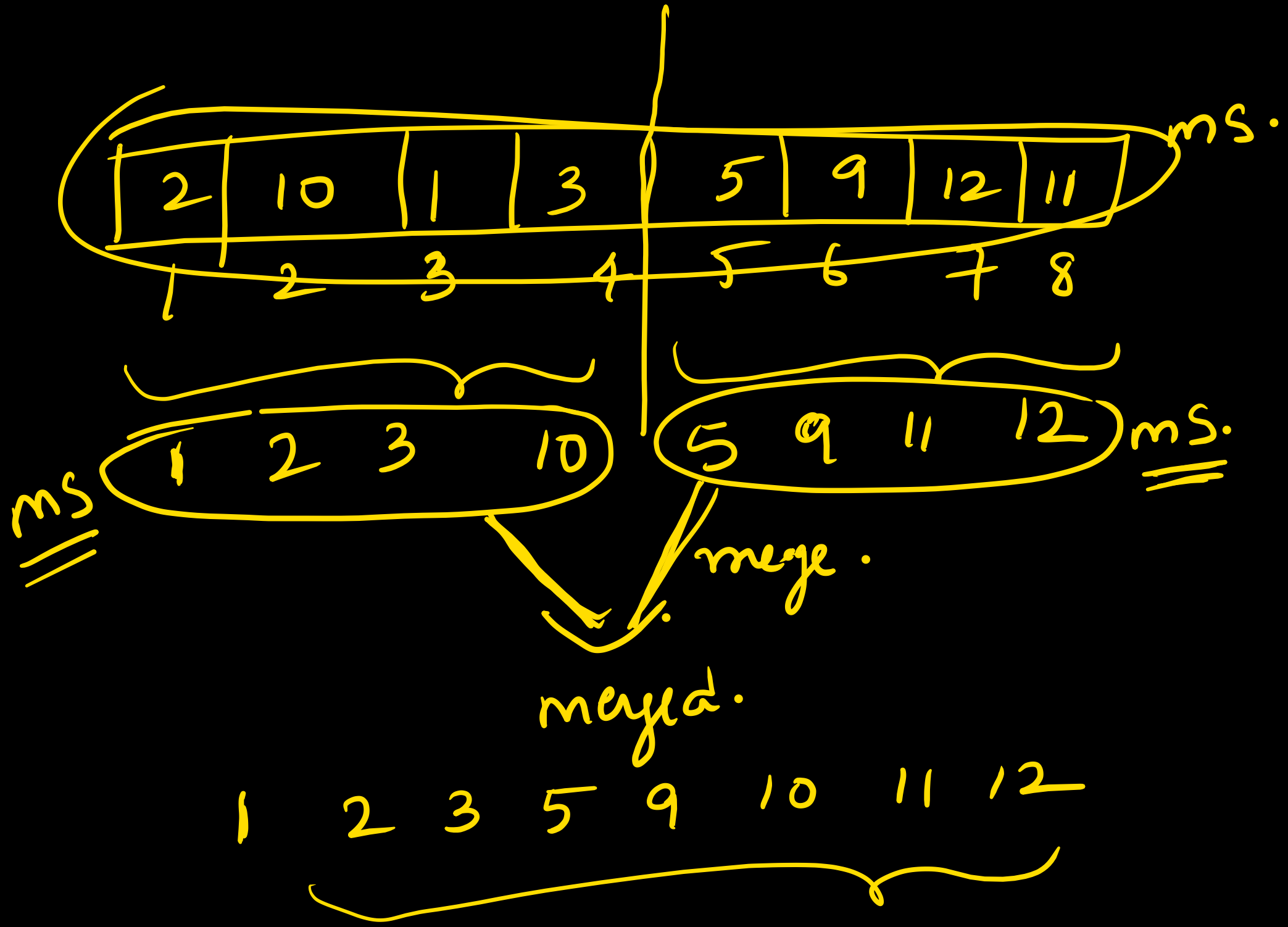
≤ 1
 $\uparrow$

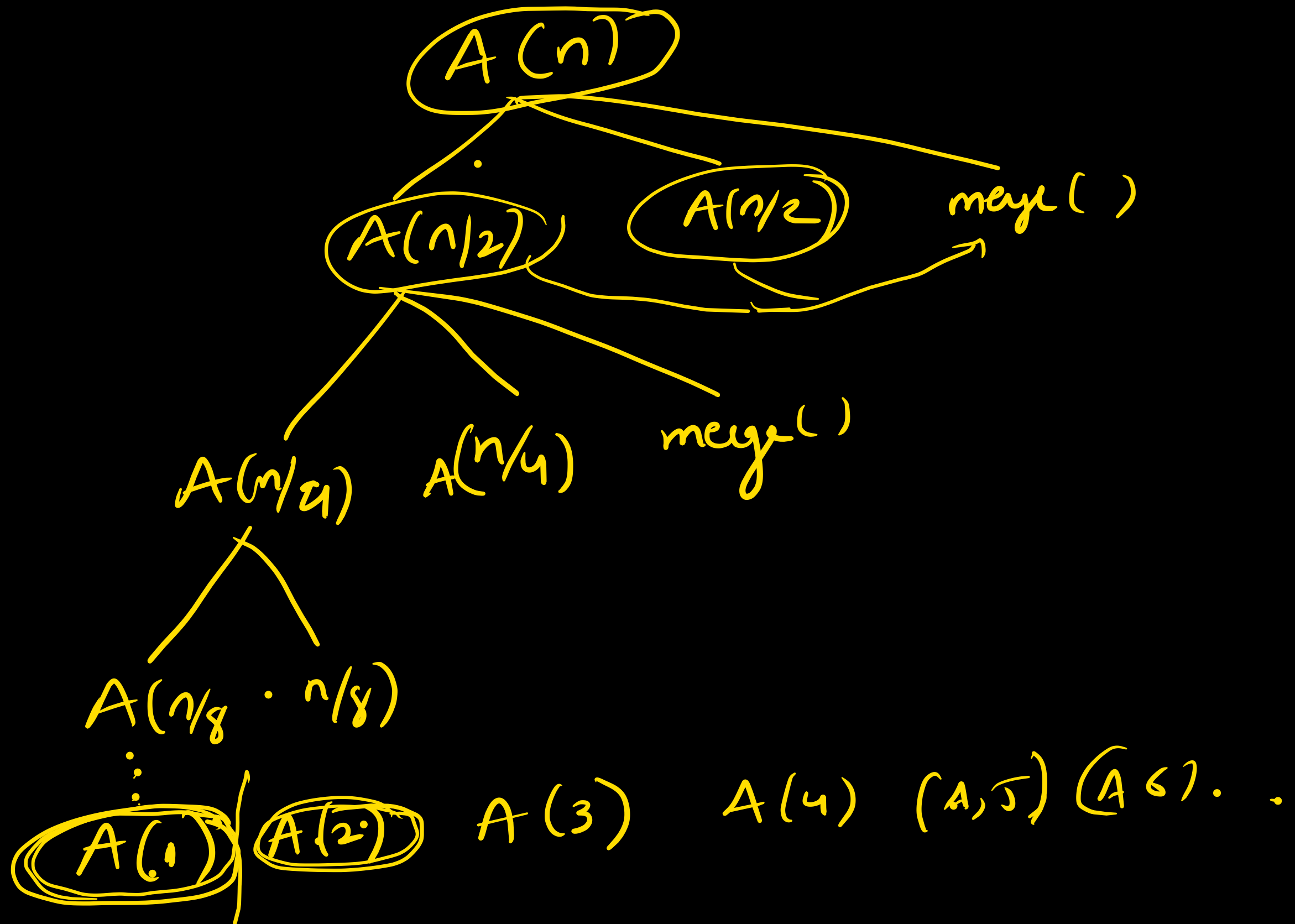
{
 then $q = \lfloor \frac{p+r}{2} \rfloor$ $\rightarrow$ divide.
 merge_sort (A, p, q)
 merge_sort (A, q+1, r)
 merge (A, p, q, r) $\rightarrow$ combine.
}

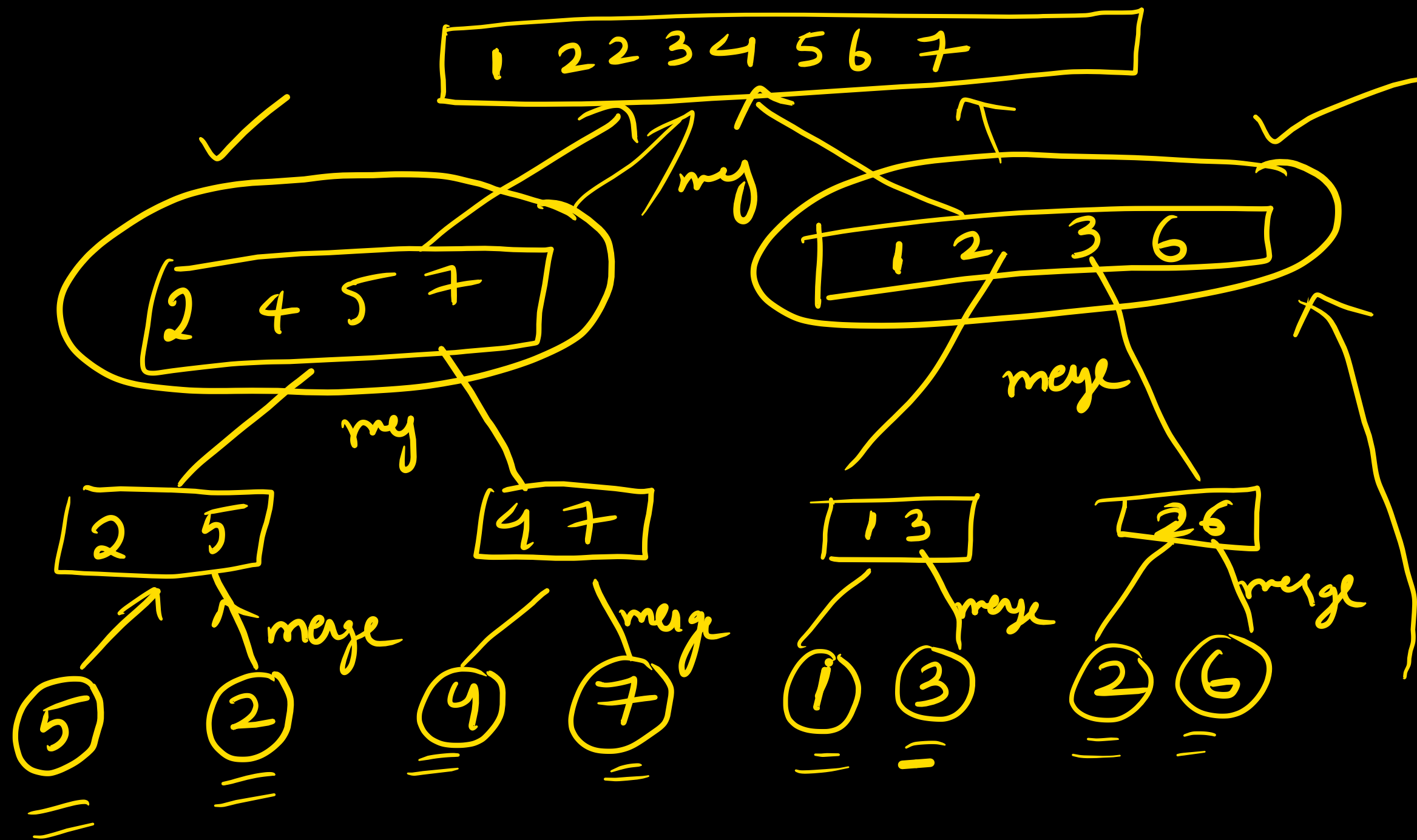
We divide the problem into problems of size 1. A single element is already sorted.

heart of ms.

}

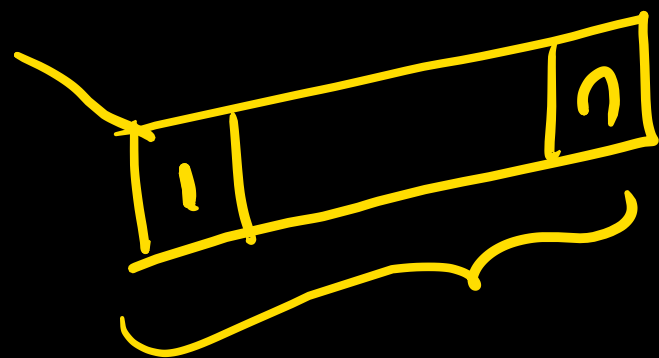
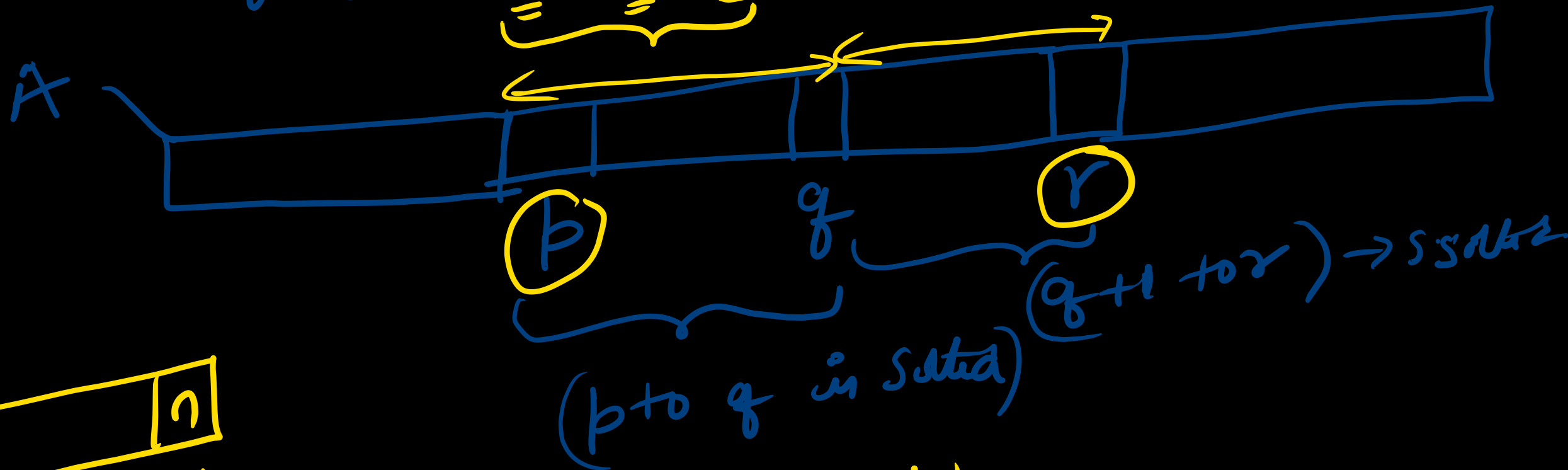






Divide the elements into one sized array and merge them
bottom up.

merge(A, p, q, r)



$A[p, q] \rightarrow$ sorted

$A[q+1, r] \rightarrow$ sorted.

we have to make $A[p, r]$ sorted

merge(A, p, q, r)

{ $n_1 = q - p + 1$ // I sorted list
 $n_2 = r - q$ // II sorted list



Create two arrays $L[1 \dots n_1 + 1]$ $R[1 \dots n_2 + 1]$

for $i = 1$ to n_1
do $L[i] = A[p + i - 1]$ // Copying $A[p] \dots [q]$ into L

for $j = 1$ to n_2
do $R[j] = A[q + j]$ // $A[q+1] \dots [r]$ into R



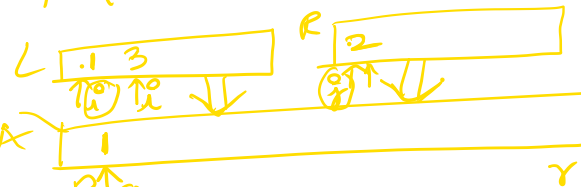
$L[n_1 + 1] = \infty$

$R[n_2 + 1] = \infty$

$i = 1$

$j = 1$

for $k = p$ to r



for $k = p$ to r

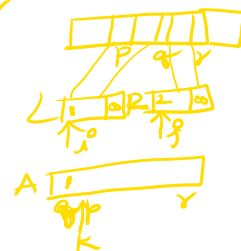
do if $L[i] \leq R[j]$

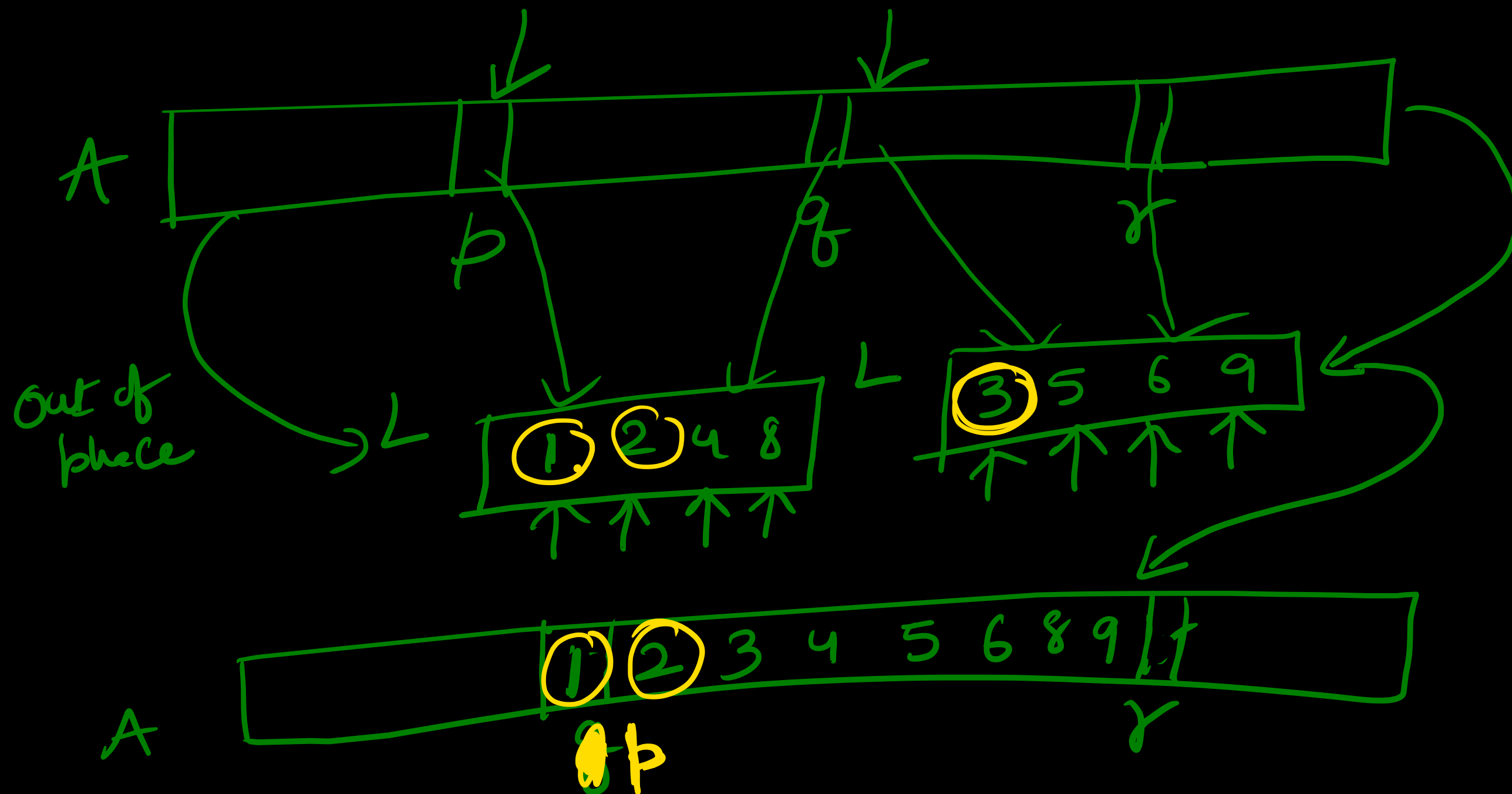
then $A[k] = L[i]$

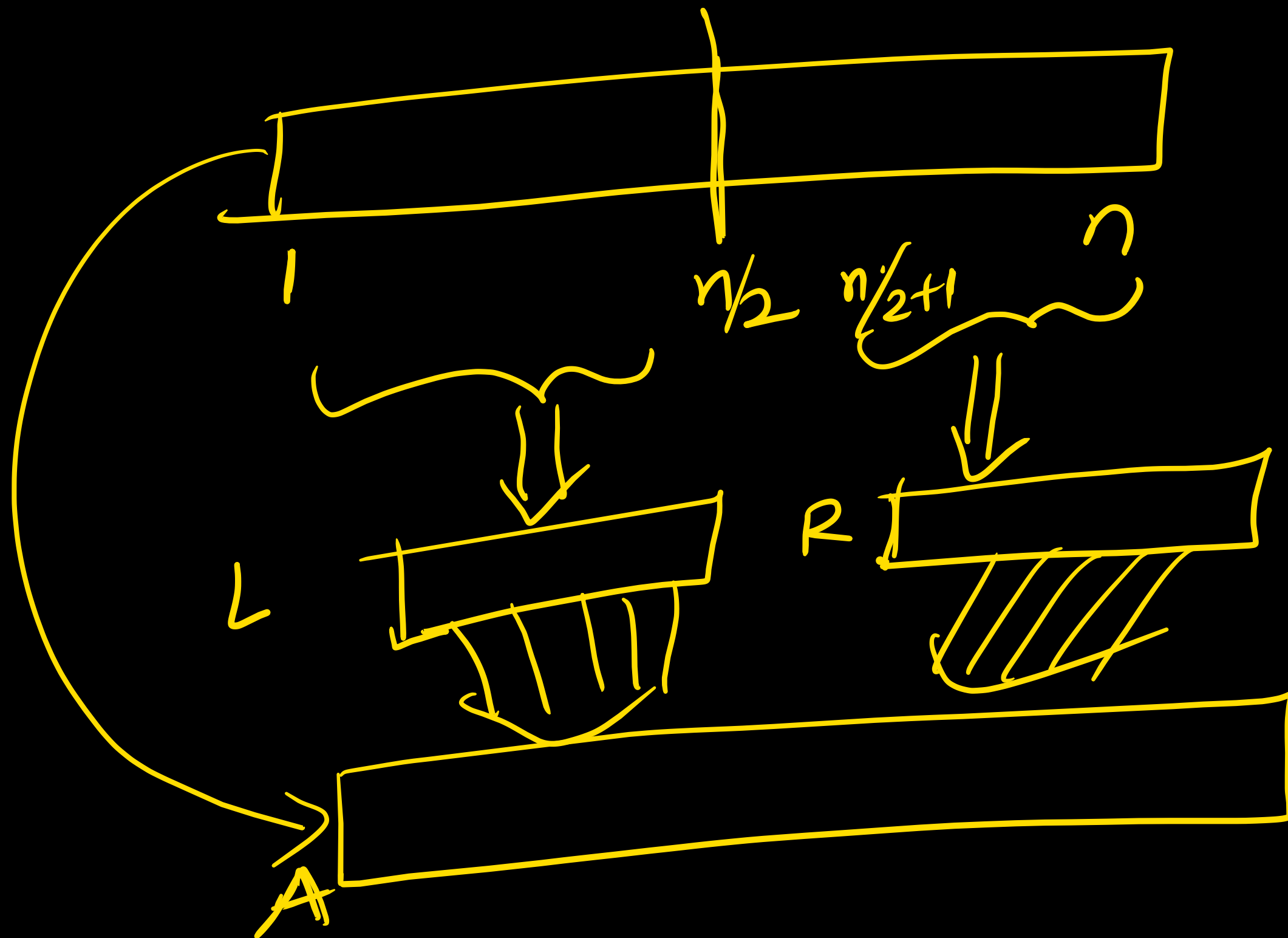
$i = i + 1$

else $A[k] = R[j]$

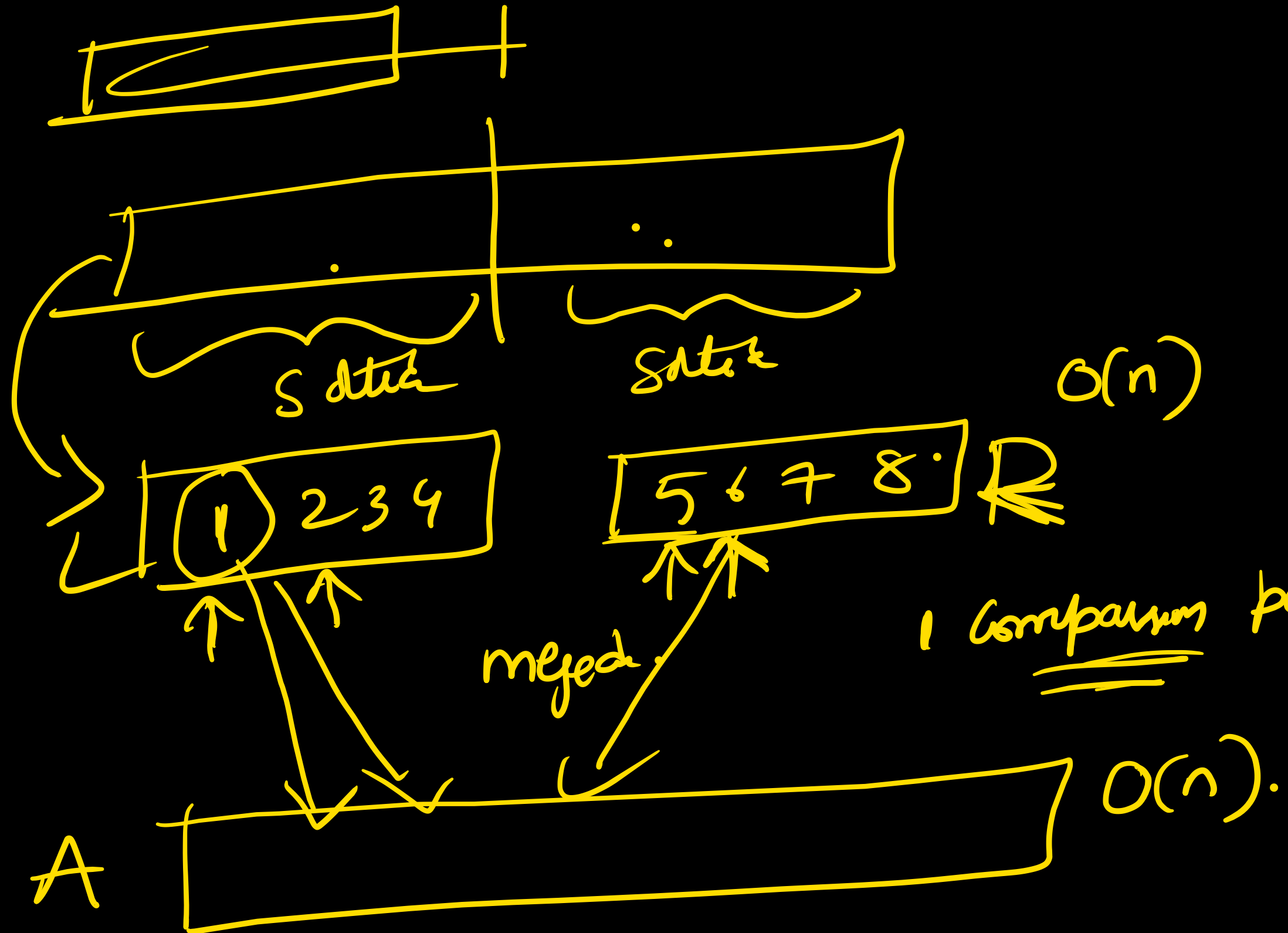
$j = j + 1$







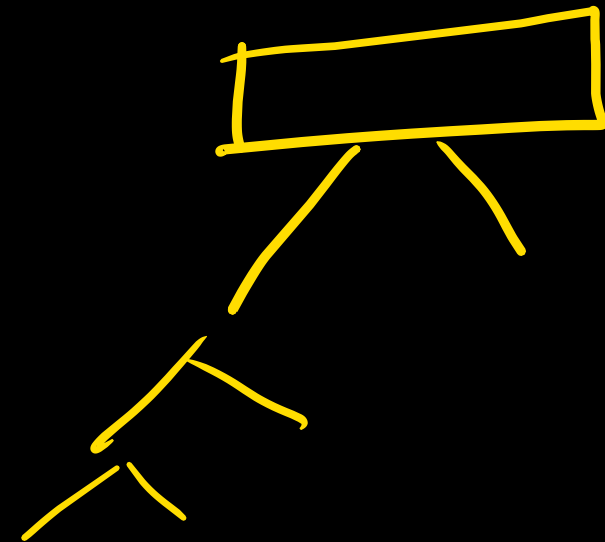
TC
to copy



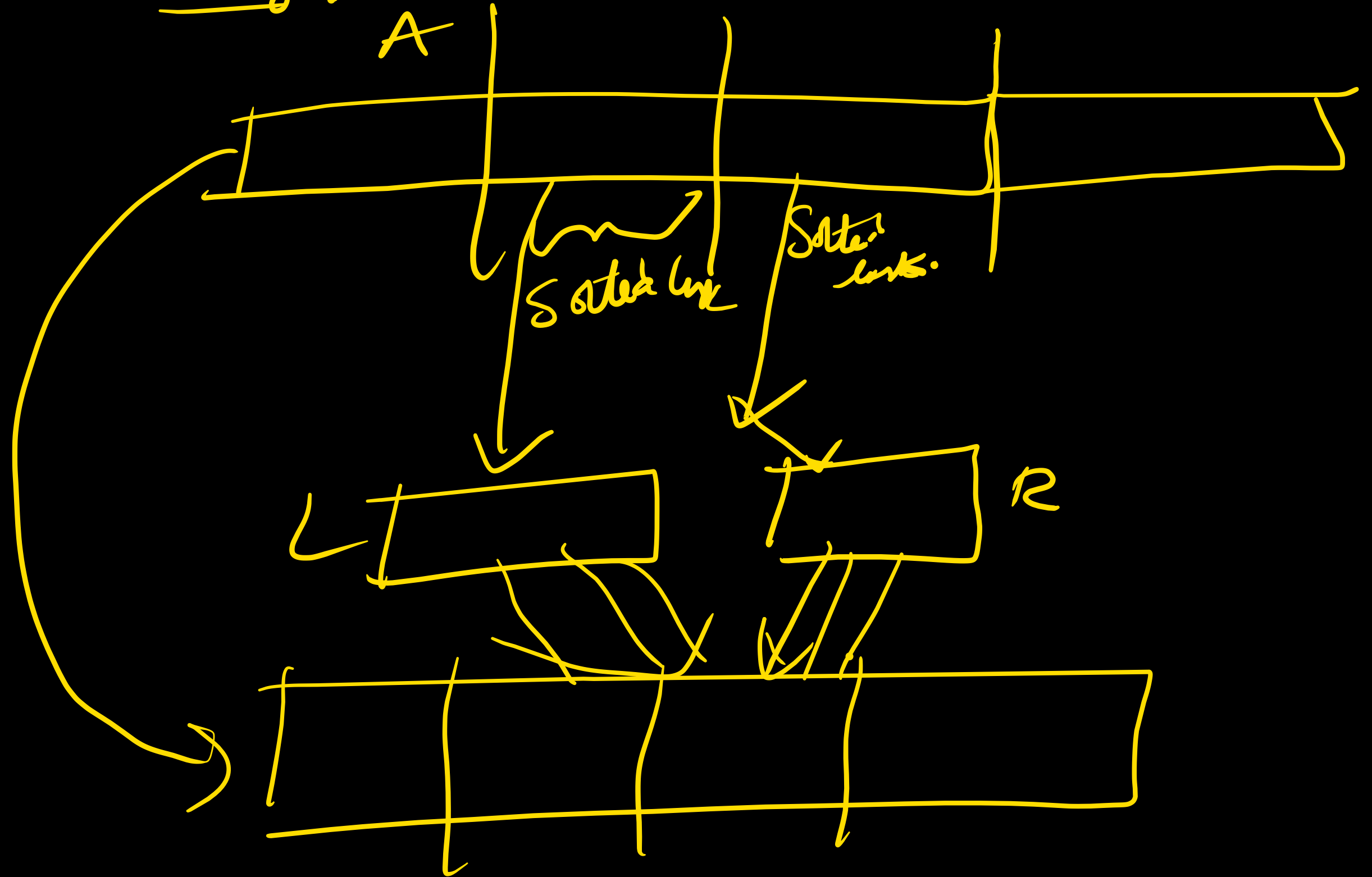
1 comparison per element.

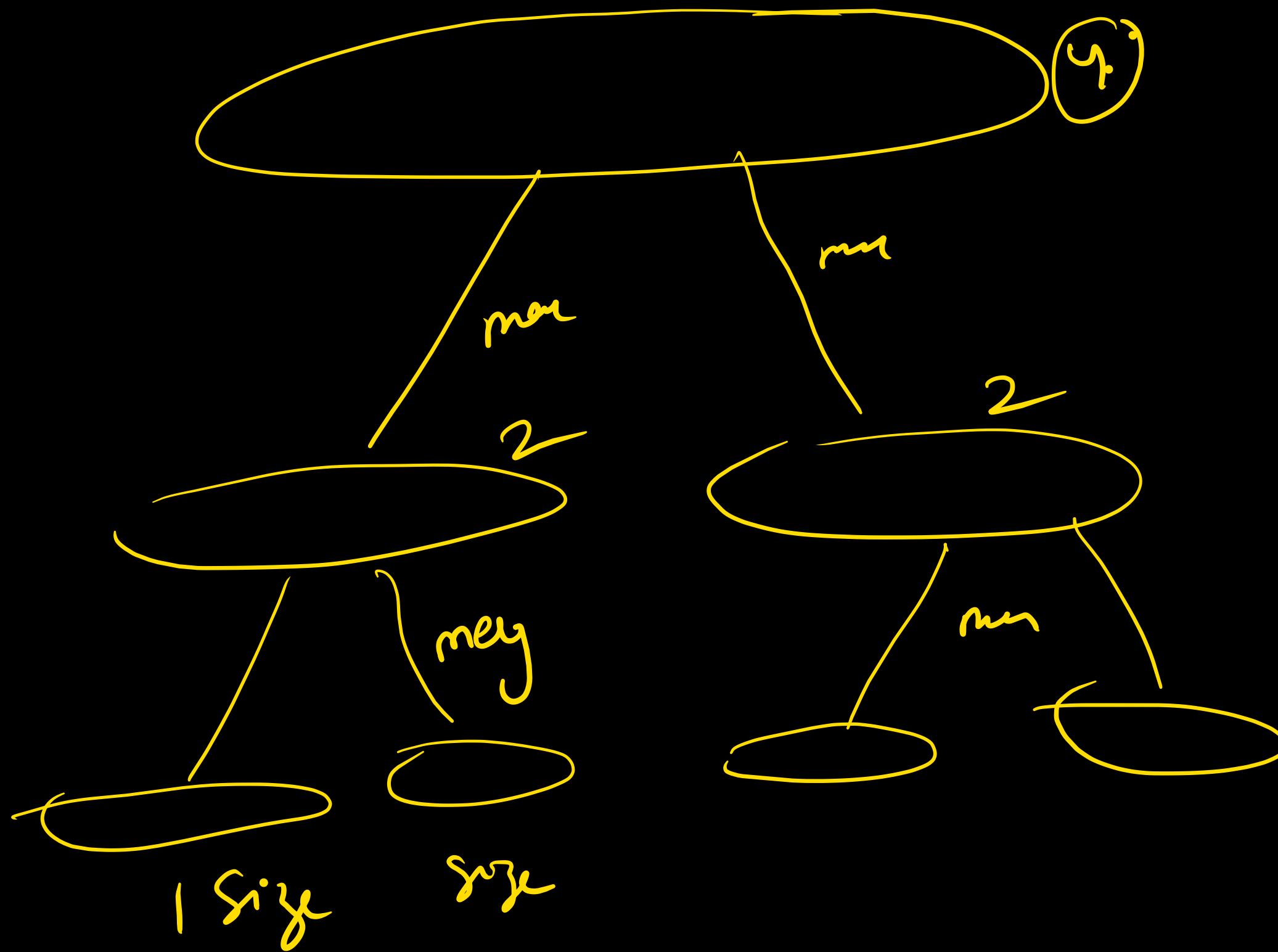
merge $\Rightarrow$ Copying from A to L and R. ($O(n)$)
 $\Rightarrow$ merging L and R into A. ($O(n)$)

merge $\Rightarrow$ $O(n)$ ✓



merge:





$$T(n) = 2T(n/2) + O(n)$$

$$T(n) = 2T(n/2) + cn$$

$$n \log_2 a$$

$$n \log_2 2$$

$$f(n)$$

$$cn$$

$$T(n) = O(n \log n)$$

$$ms(\underline{n}) \Rightarrow \underline{\underline{T(n)}}$$

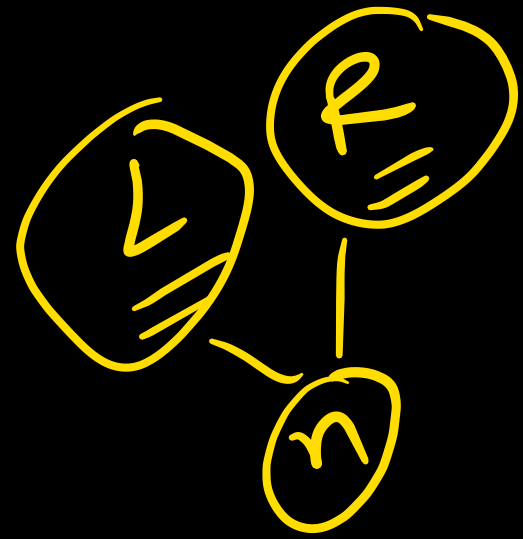
$$ms(n/2) \Rightarrow T(n/2)$$

$$ms(n/2) \Rightarrow T(n/2)$$

merge (two ~~sorted~~ sorted lists)

$$O(n) + O(n)$$

SC:::



$O(n \times \log n)$
 $O(n)$



$\therefore mS(\frac{n}{2^k})$

$m(s)$
 m

$m(n/4)$

$m(n/2)$

$m/m(n/4)$

$m(n)$

$m(n/2)$

$meye$

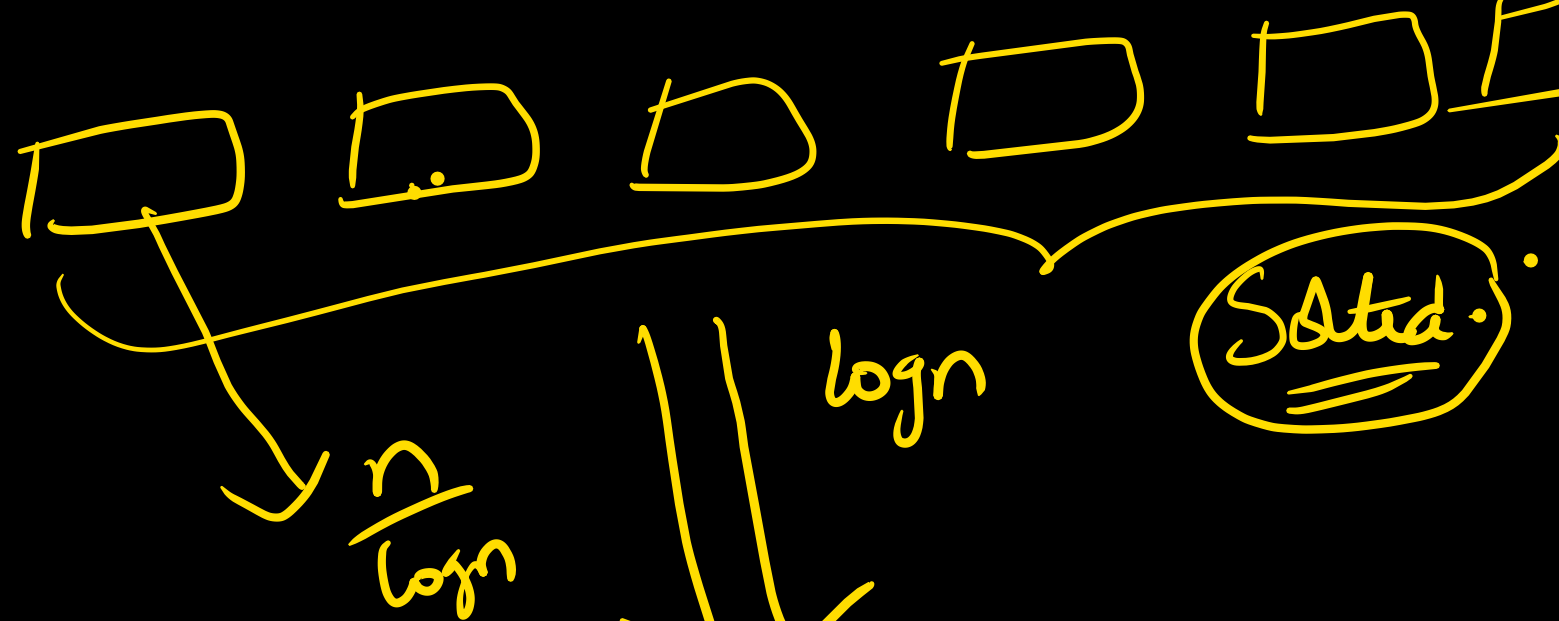
$$\frac{n}{2^k} = 1$$
$$k = \log n$$

(i/p): $\log n$ sorted subarrays each of size $\frac{n}{\log n}$

(o/p): one n element single sorted array

TC = ? ✓

$$\log n \times \frac{n}{\log n} = \underline{\underline{n}}$$



n element sorted.



$$\frac{\log n}{2^k} = 1 \Rightarrow 2^k = \log n \Rightarrow k = \log \log n$$

$$\log n / 2^2$$

$$\log n / 2$$

$$\log n \text{ bits}$$

$$O(n)$$

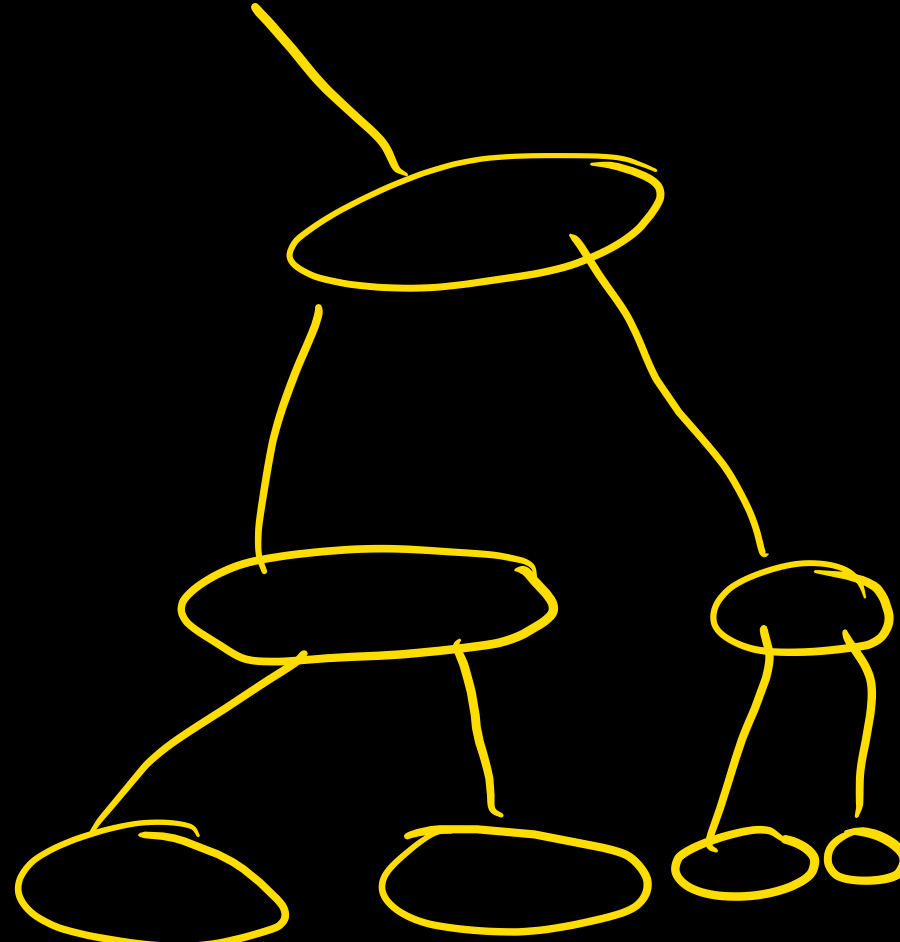
sorted

sorted

$$\frac{\log n}{2^k} = 1$$



$$O(n(\log \log n))$$



i/p: 'a' sorted subarrays each of size b

o/p: Find single sorted array

TC=?

$O(n) \nearrow a/2$
 $O(n) \nearrow a/4$
 $O(n) \nearrow a/8$
 $O(n) \nearrow a/2^k$

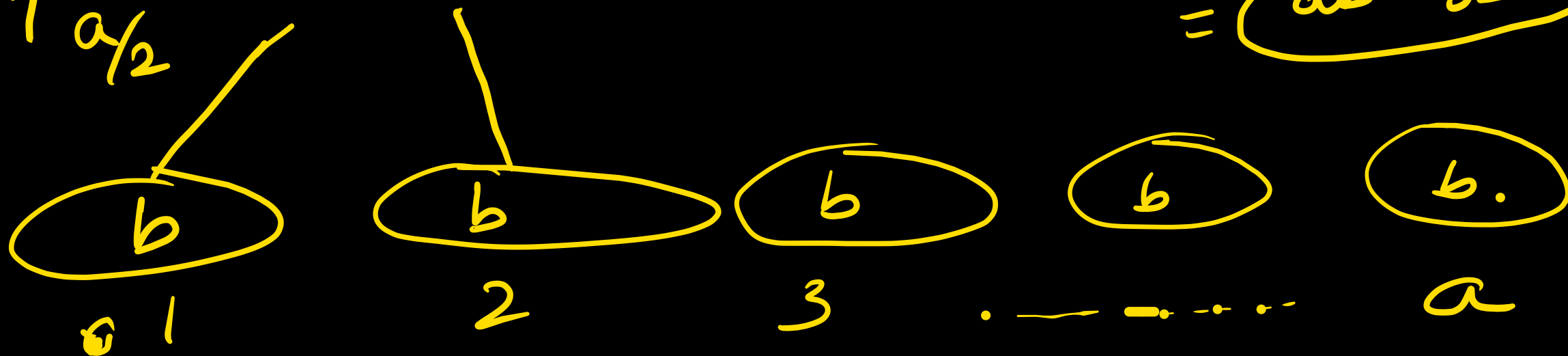
$$a/2^k = 1$$

$$a = 2^k$$

$$\Rightarrow k = (\log a)(ab)$$

$$= ab \log_2 a$$

ans.



Given two sorted arrays, TC to find $A \cap B$.

1	2	4	5	6
↑	↑ x	↑ x	↑	↑
i	i	i		

2	3	4	7	8
↑ x	↑ x	↑	↑	
j	j	j		

(2 4)

= $O(n)$